

Simulace útoků na hesla

BPC-AKR

Zdeněk Ondra, Lukáš Makovička, Jiří Pokluda

1. Obsah

1. OBSAH	2
2. ÚVOD	3
3. CÍLE PROJEKTU	4
4. TEORETICKÁ ČÁST	5
4.1. Bezpečné ukládání hesel	5
4.2. Hashovací funkce	5
4.3. Solení hesel.....	5
4.4. Metody útoků na hesla	5
4.5. Složitost hesel a entropie	6
4.6. Rizika a jejich mitigace	6
4.7. Shrnutí	6
5. REALIZACE PROJEKTU	7
5.1. Architektura řešení	7
5.2. Popis vstupů, výstupů a potřebných souborů	7
Vstupní soubory:	7
Soubory wordlist.txt a passwords_sample.db lze změnit při spuštění	7
Výstup:	7
Všechny výsledky jsou vypisovány průběžně na standardní výstup. Výstup je strukturován do pěti sekcí: porovnání hashů, slovníkový útok, brute-force, rainbow tables, analýza složitosti.	7
5.3. Seznam použitých externích knihoven.....	7
5.4. Popis implementace algoritmů.....	8
5.4.1 Slovníkový útok.....	8
Pro každý záznam v databázi postupně hashuje kandidátní hesla ze slovníku a porovnává výsledek s uloženým hashem. Útok respektuje algoritmus a sůl uloženou v záznamu.	8
5.4.2 Útok hrubou silou	8
5.4.3. Rainbow tables	8
5.3. Diagram architektury	8
6. TESTOVÁNÍ A VÝSLEDKY	9
6.1. Testovací metodika	9
6.2. Výsledky.....	9
6.3. Závěr testování	10

7. PŘEDSTAVENÍ AUTORŮ	10
8. ZÁVĚR	11
9. LITERATURA	12

2. Úvod

Projekt se zaměřuje na analýzu bezpečnosti uživatelských hesel prostřednictvím simulace různých typů útoků. V současnosti patří hesla stále mezi nejpoužívanější autentizační mechanismy, je proto důležité porozumět jejich zranitelnostem a metodám, kterými mohou být kompromitována.

Cílem projektu je vytvořit aplikaci schopnou provádět útoky na hashovaná hesla a vyhodnocovat jejich odolnost vůči prolomení. Projekt kombinuje teoretickou analýzu bezpečného ukládání hesel s praktickou implementací útočných technik.

3. Cíle projektu

Hlavním cílem projektu je vytvořit nástroj, který dokáže simulovat různé typy útoků na hashovaná hesla a následně vyhodnotit, jak jsou tato hesla odolná vůči prolomení.

V rámci projektu budou implementovány tři základní metody útoků. Slovníkový útok, který využívá předpřipravené seznamy běžně používaných hesel. Útok hrubou silou, který systematicky zkouší všechny možné kombinace znaků. A zjednodušená implementace rainbow tables, tedy předpočítaných tabulek hashů, které umožňují rychlejší prolomení na úkor velikosti tabulek.

Kromě samotných útoků se projekt zaměří také na porovnání různých hashovacích funkcí z hlediska jejich odolnosti. Součástí bude i vyhodnocení toho, jak doba prolomení hesla závisí na jeho složitosti, tedy na délce, použitých znacích a mechanismech a celkové entropii.

Vedle těchto hlavních cílů bychom rádi vytvořili i uživatelsky přívětivé rozhraní pro snadné testování, doplněné o vizualizaci výsledků a statistik. Projekt by měl také prakticky demonstrovat, proč je důležité používat solení a moderní hashovací funkce.

4. Teoretická část

4.1. Bezpečné ukládání hesel

Hesla nesmí být nikdy ukládána v čistém textu, protože v případě kompromitace databáze by útočník okamžitě získal přístup k uživatelským účtům. Podle doporučení OWASP [1] je nutné používat jednosměrné hashovací funkce, doplněné o další ochranné mechanismy.

Základní principy bezpečného ukládání hesel zahrnují

- hashování, tedy jednosměrnou transformaci hesla
- solení, což znamená přidání náhodných dat k heslu před hashováním
- použití výpočetně náročných a pomalých hashovacích funkcí, které zpomalují útoky

4.2. Hashovací funkce

Hashovací funkce převádějí vstupní data na pevně dlouhý řetězec. Pro ukládání hesel je zásadní, aby byly odolné vůči pokusům o zpětný výpočet a zároveň dostatečně pomalé.

Funkce jako MD5 a SHA-256 jsou sice kryptografické hashovací funkce, ale jsou velmi rychlé, což je činí nevhodnými pro ukládání hesel [4]. Útočníci mohou díky GPU akceleraci testovat obrovské množství kombinací za krátký čas.

Naopak bcrypt byl navržen přímo pro hashování hesel [2]. Obsahuje integrované solení a umožňuje nastavit tzv. cost faktor, který určuje výpočetní náročnost. Díky tomu je odolnější vůči moderním útokům a lépe se přizpůsobuje rostoucímu výkonu hardware [5].

4.3. Solení hesel

Solení spočívá v přidání unikátní náhodné hodnoty ke každému heslu před jeho hashováním. Tím se zajišťuje, že i stejná hesla mají odlišné hashe.

Hlavní výhodou solení je ochrana proti předpočítaným útokům, zejména rainbow tables. Bez soli by bylo možné použít již existující databáze hashů a výrazně urychlit prolomení hesel [1].

4.4. Metody útoků na hesla

Existuje několik základních metod útoků na hesla, které se liší svou efektivitou a náročností.

Slovníkový útok využívá seznamy běžných hesel a jejich variant. Je velmi efektivní proti slabým heslům a může prolomit přibližně 30 % hesel [3].

Útok hrubou silou systematicky zkouší všechny možné kombinace znaků. Je univerzální, ale jeho časová náročnost roste exponenciálně s délkou hesla. Pro heslo délky n ze znakové sady o velikosti k je potřeba až k^n pokusů [4].

Rainbow tables představují předpočítané tabulky hashů, které umožňují rychlé vyhledání hesel bez nutnosti opakovaného hashování. Jejich nevýhodou je vysoká paměťová náročnost a neefektivita při použití solení [1].

4.5. Složitost hesel a entropie

Bezpečnost hesla lze vyjádřit pomocí entropie, která udává množství informace v hesle. Pro heslo délky n ze znakové sady o velikosti k platí:

$$E = \log_2(k^n)$$

Například heslo o délce 8 znaků

- pouze malá písmena (26 znaků) má entropii přibližně 37,6 bitů
- alfanumerické znaky a symboly (~94 znaků) mají entropii přibližně 52,4 bitů

Vyšší entropie znamená vyšší odolnost vůči útokům hrubou silou [4].

4.6. Rizika a jejich mitigace

Bezpečnost hesel je ovlivněna jak kvalitou hesel, tak způsobem jejich ukládání. Nesprávným nakládáním s hesly vzniká několik zásadních bezpečnostních rizik [1][5]. Mezi hlavní rizika patří krátká hesla, absence solení, použití nevhodných hashovacích algoritmů.

Tato rizika lze zmírnit kombinací následujících opatření

- zavedení pravidel pro silná hesla (zejména délka, pak kombinace znaků)
- použití unikátní soli pro každé heslo
- využití moderních hashovacích funkcí (např. bcrypt)
- omezení počtu pokusů o přihlášení a detekce útoků
- zákaz běžných hesel a edukace uživatelů
- případné využití vícefaktorové autentizace

4.7. Shrnutí

Bezpečné ukládání hesel vyžaduje kombinaci více přístupů. Samotné hashování nestačí, klíčovou roli hraje solení, volba vhodného algoritmu a ochrana proti útokům. Nejvyšší úroveň bezpečnosti lze dosáhnout kombinací silných hesel, pomalých hashovacích funkcí a vhodně navržených ochranných mechanismů.

5. Realizace projektu

5.1. Architektura řešení

Aplikace je implementována v jazyce Python a je rozdělena do modulů. Vstupním bodem je skript `main.py`, který orchestruje spouštění všech simulačních scénářů.

Moduly:

`src/hashers.py` – implementace hashování s i bez soli pro všechny zmíněné algoritmy

`src/database.py` – vytváření, ukládání a načítání záznamů z databáze hesel

`src/attacker.py` – implementace slovníkového útoku a útoku hrubou silou

`src/rainbow.py` – sestavení rainbow table a vyhledávání pomocí redukčních funkcí

`src/analyzer.py` – měření rychlosti hashování, výpočet entropie, formátování výstupů

5.2. Popis vstupů, výstupů a potřebných souborů

Vstupní soubory:

`wordlist.txt` – slovník nejčastějších hesel

`passwords_sample.db` – databáze hashů hesel ve formátu `username:algo:salt:hash`

Soubory `wordlist.txt` a `passwords_sample.db` lze změnit při spuštění.

Výstup:

Všechny výsledky jsou vypisovány průběžně na standardní výstup. Výstup je strukturován do pěti sekcí: porovnání hashů, slovníkový útok, brute-force, rainbow tables, analýza složitosti.

5.3. Seznam použitých externích knihoven

`bcrypt` – hashování hesel algoritmem `bcrypt`

`hashlib` – `md5`, `sha-1`, `sha-256`, `sha-512`, `pbkdf2-hmac`

`itertools` – generování kombinací pro brute-force modul

`os`, `time`, `math`, `string` – systémové operace, měření času, matematické výpočty

5.4. Popis implementace algoritmů

5.4.1 Slovníkový útok

Pro každý záznam v databázi postupně hashuje kandidátní hesla ze slovníku a porovnává výsledek s uloženým hashem. Útok respektuje algoritmus a sůl uloženou v záznamu.

5.4.2 Útok hrubou silou

Systematicky generuje všechny kombinace znaků z definovaného znakového prostoru od délky 1 do max_length. Průběh je průběžně vypisován na terminál. Počet kombinací roste exponenciálně: pro 36 znaků a délku 8 je to $2,8 \times 10^{12}$ kombinací.

5.4.3. Rainbow tables

Tabulka se skládá z řetězců, kde se střídají operace hash a redukce:

$p_0 \rightarrow (\text{hash}) \rightarrow h_0 \rightarrow (\text{reduce}_0) \rightarrow p_1 \rightarrow (\text{hash}) \rightarrow h_1 \rightarrow (\text{reduce}_1) \rightarrow \dots \rightarrow p_n$

Uloženy jsou pouze počáteční a koncové hodnoty každého řetězce. Při útoku se hledaný hash zpětně prochází řetězcem a hledá se shoda s koncem uloženého řetězce. Funguje výhradně na nesolených hashích.

5.3. Diagram architektury

(Sem vložte blokový diagram – viz níže)

main.py (orchestrátor)

```
└─ src/hashers.py    ← hash_password(), verify_password()
└─ src/database.py   ← create_entry(), load_database()
└─ src/attacker.py   ← dictionary_attack(), brute_force_attack()
└─ src/rainbow.py     ← build_table(), lookup(), rainbow_attack()
└─ src/analyzer.py   ← measure_hash_speed(), password_entropy()
```


6. Testování a výsledky

6.1. Testovací metodika

Testování probíhalo na počítači s procesorem Apple M-series (ARM64), macOS, Python 3.14. Čas je měřen funkcí `time.time()`. Pro každou metodu útoku byl vytvořen set testovacích záznamů s definovanými hesly.

6.2. Výsledky

Porovnání hashovacích funkcí:

Algoritmus	Čas / hash	Hash / s	Hodnocení
MD5	0.002 ms	449 261	nevhodný
SHA-1	0.002 ms	472 971	nevhodný
SHA-256	0.002 ms	475 221	bez soli nevhodný
SHA-512	0.002 ms	453 242	bez soli nevhodný
bcrypt	70.0 ms	14	výborný
PBKDF2-SHA256	13.9 ms	72	Velmi vhodný

Slovníkový útok (wordlist.txt, 157 hesel):

Algoritmus	Prolomeno	Rychlost
SHA-256 + sůl	10/10	0.1 ms
Bcrypt	3/3	917 ms

U reálně použitelného slovníku s např. 14 miliony hesly by útok na SHA-256 trval přibližně 16 sekund, na bcrypt přes 11 dní.

Útok hrubou silou (a-z + 0-9, max. délka = 4):

Algoritmus	Prolomeno	Rychlost
MD5	6/6	5.4 ms
SHA-256	6/6	4.9 ms
bcrypt	6/6	5.6 min

Hesla byla záměrně krátká. Výsledek ale ilustruje cca 60 000x pomalejší zpracování bcrypt oproti ostatním algoritmům. Při delším hesle by byl čas znatelně delší. Prodloužení hesla o jeden znak zvýší počet kombinací 36×, bcrypt navíc přidává fixní multiplikátor cca 60 000× oproti MD5.

Rainbow tables (MD5, délka = 3, malá písmena, 1500 řetězců):

Prolomeno	Čas sestavení
6/8	2.07 s

Dvě hesla nebyla nalezena kvůli kolizím při sestavování tabulky.

Analýza složitosti hesel:

Heslo	Délka	Entropie	MD5	bcrypt
ab	2	9.4 b	1.5 ms	47 s
abc	3	14.1 b	39 ms	20 min
password	8	37.6 b	5.4 dní	464 let
P@ssw0rd	8	52.4 b	430 let	13.5 mil. Let
zatrix-5hetpu-fucQek-e5lk7a	28	164 b	$1,7 \times 10^{36}$ let	prakticky nikdy

6.3. Závěr testování

Výsledky potvrzují, že bcrypt výrazně prodlužuje dobu prolomení i u slabých hesel. Sůl znemožňuje použití rainbow tables. Délka hesla má exponenciální vliv na bezpečnost.

7. Představení autorů

Zdeněk Ondra se věnuje především návrhu architektury aplikace a implementaci jednotlivých útoků. Současně se podílí na návrhu struktury projektu a integraci jednotlivých modulů do funkčního celku.

Lukáš Makovička se soustředí na práci s hashovacími funkcemi, jejich porovnání a zpracování testovacích dat. Dále se podílí na návrhu způsobu ukládání hashů a implementaci podpory solení.

Jiří Pokluda má na starosti vyhodnocení výsledků, návrh metrik a analýzu bezpečnosti hesel. Zároveň se věnuje interpretaci naměřených dat a jejich prezentaci ve formě přehledných výstupů a podílí se na implementaci.

Na teoretické části a finální podobě projektu se podílejí všichni autoři společně.

8. Závěr

Projekt úspěšně splnil všechny stanovené cíle. Byla implementována aplikace v jazyce Python 3 simulující tři metody útoku na hashovaná hesla – slovníkový útok, útok hrubou silou a útok pomocí rainbow tables – pracující se šesti hashovacími algoritmy (MD5, SHA-1, SHA-256, SHA-512, bcrypt, PBKDF2).

Měření prokázala zásadní rozdíly mezi jednotlivými algoritmy. Rychlé funkce jako MD5 a SHA-256 umožňují útočníkovi otestovat stovky tisíc kombinací za sekundu, zatímco bcrypt je záměrně až 33 000× pomalejší, což offline útoky výrazně prodražuje.

Slovníkový útok odhalil, že běžná hesla jsou prolomitelná v řádu milisekund bez ohledu na použitý algoritmus. Rainbow tables demonstrovaly efektivitu předpočítaných struktur na nesolených hashích a zároveň ukázaly, proč unikátní sůl tento typ útoku prakticky znemožňuje. Analýza složitosti hesel potvrdila exponenciální závislost doby prolomení na délce hesla a velikosti znakového prostoru.

Na základě získaných výsledků lze formulovat jednoznačné doporučení: pro bezpečné ukládání hesel je nezbytné kombinovat pomalou hashovací funkci (bcrypt nebo PBKDF2), unikátní kryptografickou sůl pro každé heslo a dostatečnou složitost hesla. Použití MD5 nebo SHA bez soli je z bezpečnostního hlediska zcela nevhodné.

9. Literatura

- [1] OWASP Foundation. *Password Storage Cheat Sheet*. Dostupné z: [https://cheatsheetseries.owasp.org/cheatsheets/Password Storage Cheat Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)
- [2] PROVOS, Niels; MAZIÈRES, David. *A Future-Adaptable Password Scheme (bcrypt)*. 1999. Dostupné z: <https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf>
- [3] BONNEAU, Joseph. *The science of guessing: analyzing an anonymized corpus of 70 million passwords*. IEEE Symposium on Security and Privacy, 2012.
- [4] STALLINGS, William. *Cryptography and Network Security: Principles and Practice*. 7th ed. Pearson, 2017. ISBN 978-0134444284.
- [5] NIST. *Digital Identity Guidelines – Authentication and Lifecycle Management (SP 800-63B)*. Dostupné z: <https://pages.nist.gov/800-63-3/sp800-63b.html>